Routing and published responses

Use this reference for the detailed contract or recovery needed by the current operation.

- Published routes
- Platform HTTP behavior

Published routes

`routes` is the optional published-routing domain. It does not configure the development Preview. `PUT <config-plane>/config/routes` replaces this domain with its complete fragment; its stored declaration is supplied for the next publish. Unknown transport details belong to configuration.

Rule schema

JSON

```
{"routes":[{"path":"/api/*","target":"server"},{"path":"/assets/*","target":"st
```

Field	Accepted declaration
`path`	1–256 characters; starts with `/`; segments use `A-Z a-z 0-9 . _ ~ -`; optional trailing `/*`
`target`	`"server"` or `"static"`
`cache`	Optional. Static: `"immutable"`, `"no-cache"`, or integer seconds 0–31536000. Server: `"no-cache"` or integer seconds 0–600.
`spaFallback`	Optional boolean, static target only
Rule collection	1–32 rules in declaration order

Each path segment must contain at least one non-dot character. Bare `/`, `/.`, `/..`, empty segments, a trailing `/`, a mid-path wildcard, query/fragment delimiters, and whitespace are invalid. `/*` is the all-path wildcard. The declaration model is ordered, first match wins; duplicate paths and a rule fully covered by an earlier wildcard are rejected.

`server` identifies the application server and requires `features.server: true`. `static` identifies published build output and requires a `build` contract. Both are declaration-time

prerequisites. An empty array is invalid; a domain fragment `{}` removes the `routes` key and returns to the platform's default declaration state.

Reserved paths

A literal declaration at or below these paths is refused:

Plain Text

```
/favicon.ico
/manus-oauth/
/api/scheduled/
/manus-storage/
/__manus/
/__system__/
/sitemap.xml
/robots.txt
```

A broader wildcard remains legal: `/api/*` is allowed, whereas `/api/scheduled/*` is not. Platform-owned paths have routing priority over the user table. Their actual handling varies;
identity, scheduled callbacks, storage objects, and SEO documents belong to their respective
capability contracts, not to a single blanket reserved-path response.

Stored rules and serving evidence

The schema has no User-Agent or arbitrary-header match condition. The table records path/target
selection, not application logic. Hybrid publishing can produce both static output and an application server, whose build inputs remain separate contracts.

The inspected overview reports no effective rule table (`atoms.routes.effective` is `null`), and publish status has no separate static-upload verdict. An accepted declaration and an aggregate successful publish are not observations of which rule a gateway applied.

The schema accepts `cache` and `spaFallback` ; their presence is not by itself evidence of a particular live cache header or miss response. Verified response behavior and its serving-path
scope are in published response handling when those details are relevant.

Validation codes

Code	Declaration rejected because
`routes_too_many`	More than 32 rules
`routes_reserved_path`	Literal path enters a platform-reserved prefix
`routes_duplicate`	Same path appears twice
`routes_unreachable`	An earlier wildcard fully covers this rule
`routes_target_needs_server`	A server target lacks the server capability
`routes_target_needs_build`	A static target lacks build output declaration
`routes_server_cache_invalid`	Server cache is `"immutable"` or exceeds 600 seconds
`routes_spa_fallback_needs_static`	SPA fallback is present on a non-static target

Other path/type/empty-array errors use the common schema-validation structure.

Route preparation and validation

Declare rules in the order their requests should match. Keep responses computed per request
on the server target; no cache value makes dynamic content suitable for static object serving.
A User-Agent distinction belongs in application server behavior, not an invented route field.
Moving a path to the server also moves ordinary human requests there, with the corresponding
runtime and hosting costs.

After a hybrid publish has cut over and the site answers, request at least one actual object for each rule intended to serve static output. Verify that the static half is present rather than accepting the aggregate publish status as its proof. If server paths work and a static sample fails, inspect the output directory/build result and the actual serving path; do not assume a particular cause from the 404 alone.

For an SPA that needs deep links, configure the supported static fallback declaration and test
representative direct-entry routes and real missing assets on the deployed site. The declared

`spaFallback` field is not evidence of its live behavior; preserve the default-path distinctions in published response handling rather than assuming all missing extensions behave alike.

When a route change affects response freshness, use the asset-cache delivery workflow in published response handling; that page owns filename/cache choices and served-header verification, while this page owns the route declaration and path checks.

Platform HTTP behavior

HTML transformation

The published gateway enhances eligible application HTML with platform-owned head metadata,
a custom element, an inline configuration script, and a CDN-loaded runtime for the badge and
editor affordances. Missing/placeholder title and absent Open Graph/canonical metadata can be
supplied; application body content is not generated.

In the backend-response path, enrichment applies to production `text/html` responses with a successful status outside `/api/`. Redirects, errors, WebSocket/101 responses, non-HTML,
and `/api/` responses bypass that enrichment. Authentication by the application is not an exemption for an otherwise eligible page. Non-production deployment responses use the header
handling without production body enrichment.

The injected element, inline content, and CDN host are platform-owned and can change. A project's CSP governs those injected scripts as well as its own. Badge visibility is an owner-side Dashboard setting; it is not a project response header, environment variable, or config-plane domain. The documented owner control for hiding it is membership-gated.

Cache behavior and static misses

The frontend/static gateway sends `Cache-Control: max-age=7776000` for static non-HTML output (90 days). HTML processed by its HTML handler instead receives `Cache-Control: no-cache, no-store, must-revalidate`, `Pragma: no-cache`, and `Expires: 0`.
That handler also adds `Strict-Transport-Security: max-age=31536000; includeSubDomains`.

The transparent `/assets/*` path, when backed by project object storage, returns stored objects

with `max-age=7776000` and a `x-manus-proxy-mode: transparent-assets/<logicVersion>` marker. It does not perform HTML enrichment on that direct object response. A missing object in that
path falls back to the application server; it is not an unconditional static 404.

The separate frontend-output path can fall back to root `index.html` for an extensionless miss and for a missing lowercase `.html` path. If neither the requested object nor the applicable fallback exists, it returns 404. These are verified default serving paths, not proof that a stored `spaFallback` declaration was consumed.

Backend responses preserve application cache headers except when they enter the eligible HTML
transformation above. The static default is not applied to arbitrary API/backend responses. A republish changes selected platform output but does not invalidate copies already held by browsers, so a stable asset URL can continue to identify a cached older response.

The cache values accepted by the route schema do not by themselves establish which response headers a deployment serves. The defaults above describe the serving paths;
this contract makes no additional per-route header or infinite-lifetime guarantee for an `immutable` declaration. The actual response identifies the applied cache policy.

Server HTML cache

A server rule's `cache: N` reuses eligible application HTML in the platform's shared edge cache
for up to N seconds. `cache: 0` and `cache: "no-cache"` disable this cache. Omitted cache keeps the previous serving behavior. Static cache values and their browser-cache semantics are unchanged.

Eligibility is checked after platform access control and before HTML enhancement:

- Only public production deployments with a known deployment ID, GET, HTTP 200 `text/html`,
  and paths outside `/api/` are eligible. Other content types, errors and redirects are not stored.
- Cookies, Authorization, Range/conditional requests, upgrades, or request cache-bypass directives
  bypass this cache. An application response with `Set-Cookie`, `private`, `no-store`, or `no-cache` prevents storage. These restrictions win over the route declaration.
- `Vary` is supported only for `Accept-Encoding`; other values, including `*`, bypass storage. The application must not share personalized or otherwise varying HTML without

declaring it.

- Keys include the project, deployment, origin server, route policy and full public URL including
  query parameters, plus accepted encoding. Once the gateway observes a new deployment,
  its requests cannot reuse the previous deployment's entries. Content edits without a new
  deployment become visible after the configured TTL. Edge locations warm independently.

HTML enhancement runs on both misses and hits, using the current platform metadata. Only the
internal stored copy has a cache lifetime. Browser-facing HTML keeps `Cache-Control: no-cache, no-store, must-revalidate`; it does not advertise shared caching to uncontrolled downstream proxies. This header therefore does not mean the platform cache missed.

For an explicitly declared server cache, `X-Manus-Cache: HIT` means the application response was reused; `MISS` means an eligible response was fetched and submitted for storage; `BYPASS`
means it was not eligible or the cache was unavailable. Storage failure must not fail the page.
WebSocket responses remain untouched.

After publication, repeat the same anonymous URL without cookies or cache-bypass headers.
Verify a HIT and fewer application requests, then verify a new deployment returns new content.
Check a logged-in visit separately to confirm that it bypasses shared caching. Cache writes are
asynchronous; concurrent cold requests can still reach the application.

Changing CSP and platform markup

A CSP must allow the platform's CDN script source and account for its inline configuration, not only application-authored scripts. Do not hardcode a CDN host from old notes.

Do not parse or build application behavior on the platform's injected element/configuration names. They are not stable application interfaces. Authentication does not exempt eligible private HTML from enrichment. When a surface must exclude injected markup, use a non-HTML

response such as JSON and a client surface under the required control; do not treat a 404 or an authenticated page as an opt-out.

Do not promise that project configuration can hide the badge, or suppress the injected markup
inside the application. Use the supported owner Dashboard setting when badge visibility is requested; its membership boundary still applies.

Asset-cache delivery workflow

For assets whose contents change between publications, prefer content-hashed filenames. If a
stable filename is necessary, use an appropriate shorter/revalidating policy and verify the actual served headers; an accepted route declaration is not enough. Do not knowingly leave changing stable-name assets behind the default long cache.

For public server-rendered HTML, use the server cache declaration. Other
server responses retain their application cache policy. For a broken/unstyled page
after republish, inspect the mix of HTML and asset versions and their cache headers before rewriting unrelated layout code. Validate the affected deployed paths and distinguish an already-cached browser copy from the new platform output.